Work flow

Addons

The 8 required add-ons (per feature)

1. **ID + Registry Entry**

- Assign `FEAT-###`, category, owner role, maturity, dependencies
- Add to a `FEATURE_REGISTRY`

2. **Contracts (Inputs/Outputs/State)**

- Inputs, outputs, state mutations, reference semantics
- “No implicit IO” declaration

3. **Errors (Taxonomy + Retryability)**

- error_codes added to `ERROR_CODE_REGISTRY`
- retryable vs non-retryable, operator action

4. **Security/Privacy/Trust**

- permissions, redaction/classification impact
- signing/trust requirements (if any)

5. **Gates + Proof**

- gates added/changed (gate_id)
- proofs required/produced (proof_type + schema)

6. **Tests**

- minimum: unit + integration + golden + adversarial
- perf test only if it’s on the critical path

7. **Observability**

- events + metrics + structured logs

- “what to look at when it breaks”

8. **Docs**

- operator doc + troubleshooting
- change log entry

That’s the minimum set that prevents broken, inconsistent features.

Axion Features List

1) Control Plane

- Control Plane Core (single source of truth for runs/stages/gates/artifacts/pins/policies)
- Run orchestration API (start/stop/resume/partial runs/stage reruns/pinned inputs)
- State store + run state machine (queued → running → gated → failed → released)
- Pipeline definition store (stage order, IO, gating points, activation rules)
- Policy engine (risk classes, hard stops, overrides, expiry)
- Workspace/project management (namespaces, org/project boundaries, permissions, keys)
- Operator actions ledger (audit: who/what/when/why; tamper-evident)
- Adapter manager (local/Replit/CI envs + capability discovery)
- Notification router (gate fail, approvals, standards/template changes)
- Quota + rate limits (per project/profile: runs/day, compute, storage)

---

2) Operator UI

- Run picker + run timeline (stages/gates)
- Gate failure viewer (rule_id, pointers, diff, evidence)
- Gate trace mode (predicate-by-predicate evaluation w/ intermediate values)
- Artifact browser (type/version/hash/path)
- Spec index viewer (screens/endpoints/entities/etc.)
- Standards viewer (resolved set + why applied)
- Standards conflict resolver (conflicts + applied resolution policy)

- Template render preview (before/after, unresolved placeholders highlight)
- “One-click re-run stage” controls (pinned versions shown)
- Export bundle button (manifest hash + contents)
- Dependency inspector (upstream/downstream refs)
- Provenance chain viewer (inputs → stages → outputs)
- Saved views + filters (domain/risk/gate/stage/owner)

---

3) Core Model & Contracts

- Unified object model: Run, Stage, Gate, Artifact, Template, Standard, Proof, ConstraintPack
- Schema registry (artifact schemas + versioning + validation policy)
- Reference semantics (ref formats, resolution rules, allowed edges)
- Deterministic ID allocation (stable IDs across runs)
- Artifact type registry (type → schema → renderers → required proofs)
- Stage IO contracts enforcement (consumes/produces validated against contract)

---

4) Pipeline Orchestration & Execution

- Partial pipeline runs (start at stage N with pinned inputs)
- Parallel gate execution (where safe)
- Fast “dry run” mode (no packaging; validate-only)
- Deterministic runtime envelope (toolchain versions, env fingerprint, reproducible run cmd)

- Hermetic stage execution (sealed environment; no ambient reads)
- Deterministic filesystem overlay (only declared output paths; undeclared writes hard-fail)
- Time/entropy controls (freeze time, seed RNG, locale/timezone/order controls)
- Input snapshotting (immutable snapshot of every consumed input, incl. resolved STDs/templates)
- Resource limits (CPU/mem/time caps; concurrency caps)
- Retry policy (bounded; transient vs deterministic classification)
- Sandbox/isolation per run (FS perms + network policy)
- Rollback to last-known-good (config pin restore)

---

5) Speed & Incrementality

- Content-hash caching (skip unchanged stages)
- Incremental rebuild (only impacted stages rerun)
- Impact analyzer ("if X changes, these stages rerun")
- Cache integrity checks (prevent stale cache poisoning)
- Shared cache option (team/CI) with trust policy
- Gate short-circuit (cheap/high-signal checks first)
- Cache ROI reporting (time/$ saved per stage)

---

6) Safety & Risk Controls

- Unknown lifecycle tracker (create → block → resolve → close)
- Hard stop policies by risk class (auth/data/integrations)

- No implicit assumptions enforcement (must declare unknowns)
- Input sanitation + redaction rules for logs
- Policy exceptions registry (override with expiry + approver + justification)
- Quarantine mode (secrets/PII found → isolate + block export)
- Redaction preview (show what will be removed before writing logs/proofs)
- Command allowlist manifest per stage (restricted commands whitelist)
- Runtime egress control (network domain allow/deny by stage/profile)

---

7) Quality & Correctness

- Coverage scoring (per SPEC_INDEX bucket + minimum thresholds)
- Coverage weighting by criticality (auth/payment/core flows)
- Referential integrity visualizer (broken ref finder)
- Template linting (structure + placeholder correctness)
- Schema-first enforcement (artifact validates before “produced”)
- Acceptance-to-test linking (orphan AC/test detection)
- Cross-pack consistency gates (UI ↔ endpoints ↔ entities ↔ auth rules)
- Canonical naming engine (normalize IDs/paths/refs)
- Duplicate concept detector (near-duplicate entities/screens/rules)
- Flow integrity checks (UI flow ↔ endpoints ↔ auth ↔ entities ↔ ACs/tests)
- Auth boundary verification (protected endpoints consistently guarded across specs/scaffolds/tests)

- Data classification propagation (PII/Restricted flows into logs/proofs/exports/UI exposure rules)
- Invariant testing obligations (spec-level invariants generate required checks/tests)

---

8) Observability & Debugging

- Stage timing + bottleneck report
- Proof ledger viewer (searchable)
- Structured error taxonomy (error_code registry)
- Run diff viewer (artifact diffs between runs)
- Semantic diff classification (format/metadata/content/breaking)
- Minimal repro exporter (smallest input set to reproduce failure)
- Stage event log (structured events: start/stop/read/write/cache-hit/gate-eval)
- Flake detector (repeat-run gates to detect nondeterminism; evidence attached)
- Causality tracing (gate failure → exact upstream artifacts + transformations)
- Bisect root-cause runner (find earliest stage introducing breaking change)
- Counterfactual runner (swap/pin STD/template versions to isolate cause)
- Forensic bundle (immutable evidence pack for audits)

---

9) Standards, Templates, and Governance

- STANDARDS_INDEX + STANDARDS_ENTRIES (registry + documents)
- CONSTRAINT_PACKS (locked rulesets: naming/routing/UI/security/etc.)
- APPLICABILITY_MAP (which STDs apply by category/type/skill/build_target)

- TEMPLATE_INDEX/REGISTRY (IDs, versions, required sections, outputs mapping)
- TEMPLATE_FILES (actual template documents/files)
- Signed/verified standards and templates (tamper detection)
- Standards composition (import/extend/precedence rules; explicit merges only)
- Standards test harness (STD ships fixtures/tests: expected violations)
- Standards impact simulation (run new STD against historical runs)
- Deprecation registry + sunset dates
- Deprecation enforcement gate (fails if expired/disabled versions used)

---

10) Collaboration & Roles

- Role-based permissions (who can lock, override, publish)
- Review workflow (approve locks; approve standards/template changes)
- Comment threads on artifacts/gate failures
- Multi-operator audit trail (who ran what, when)
- Signed “release” of a kit (approver identity recorded)
- Two-person rule for high-risk overrides (auth/data/integrations)
- Separation-of-duties enforcement (author ≠ approver where scoped)
- Review bundles (diff + evidence + gate summary packet)

---

11) Compatibility & Migration

- Compatibility matrix (which versions run together)

- Compatibility negotiation (auto-select compatible versions; fail with exact constraint conflicts)
- Auto migration hints (what to bump when a change happens)
- Schema/template migration scripts (guided)
- Migration dry-run (show rewrites + resulting hashes before applying)
- Breaking vs non-breaking classification (semantic diff)
- Blast radius analysis (what a STD/template change affects: domains/gates/outputs)
- Dep graph-based staged rollout (release channels: subset → global enablement)

---

12) Knowledge & Recovery

- Knowledge-map indexer (tagged retrieval for “stuck” moments)
- Context injection rules (when to pull which knowledge)
- Golden example library per category/type preset
- Failure playbook generator (fix steps tied to docs/gates)
- Knowledge provenance (which chunks were used)
- Knowledge drift monitor (usage/failures flag outdated chunks)
- “Stuck moment” capture (tag failure + store context bundle)
- Retrieval sandbox (preview which chunks would inject before injection)
- Golden-path replayer (known-good inputs validate pipeline health)
- Postmortem generator (timeline + root cause fields + remediation tasks)

---

13) Packaging & Distribution

- PACKAGING_MANIFEST (included items + versions/hashes)
- FOLDER_TREE_SPEC (authoritative kit structure)
- BUNDLE_OUTPUTS (zipped kits / domain packs / handoff bundles)
- Multiple kit profiles (thin vs full; agent kit vs human kit)
- Deterministic bundle naming (run_id + hash)
- Selective export (only a domain/section)
- Remote artifact store support (push/pull bundles)
- Bundle signing (signature over manifest hash)
- Attestations (gate results + toolchain versions as signed metadata)
- Split bundles (code/specs/proofs separated but hash-linked)
- Remote fetch-on-demand (thin bundle references blobs; fetch artifacts when opened)
- Bundle integrity scanner (verify manifest, signatures, hash tree, required artifacts)
- Artifact transparency log (append-only log of released bundles + hashes)

---

14) Security, Privacy, Trust, and Supply Chain

- Secrets scanner on inputs/outputs
- PII redaction for logs/proofs
- Encrypted knowledge vault option
- Least-privilege execution mode
- Content classification (public/internal/restricted) + export enforcement
- Trust policy (which signers trusted for which artifact types/profiles)

- Supply-chain integrity checks (stage binaries/scripts hash/sign verification)
- Tamper-evident proofs (proof objects hashed + referenced by gates)
- SBOM for kits + pipeline (stages/adapters/deps included)
- Provenance attestations (signed: inputs → stages → outputs + env fingerprint + gate results)
- SLSA-like build levels (profile-enforced build assurance levels)

---

15) Git / CI / PR Integration

- PR check suite (gates as PR checks; failures annotated with rule_id/pointers/repro)
- Autogenerate PRs for migrations (patch + evidence checklist)
- Release channels (dev/beta/prod) with gate strictness + signing requirements
- Release candidate objects (immutable RC bundles promoted without rebuild)

---

16) Cost / Performance Governance

- Cost ledger per run (time/compute/storage/tool calls per stage)
- Budgets by profile (block/alert when exceeded)
- Performance regression gates (baseline comparison; block threshold violations)
- Artifact bloat gate (bundle size limits per profile/domain/type)

---

17) AXION's Own Test Strategy

- Golden pipeline tests (known inputs → expected outputs/hashes/gate results)

- Gate rule fixtures/unit tests (predicate correctness)
- Adversarial packs (malformed templates, circular refs, missing proofs)
- Regression suite for constraints packs (naming/routing/UI/security)
- Repro gate (rerun in second env to enforce reproducibility guarantee)
-

Control Plane

Control Plane

- Control Plane Core (single source of truth for runs/stages/gates/artifacts/pins/policies)
- Run orchestration API (start/stop/resume/partial runs/stage reruns/pinned inputs)
- State store + run state machine (queued → running → gated → failed → released)
- Pipeline definition store (stage order, IO, gating points, activation rules)
- Policy engine (risk classes, hard stops, overrides, expiry)
- Workspace/project management (namespaces, org/project boundaries, permissions, keys)
- Operator actions ledger (audit: who/what/when/why; tamper-evident)
- Adapter manager (local/Replit/CI envs + capability discovery)
- Notification router (gate fail, approvals, standards/template changes)
- Quota + rate limits (per project/profile: runs/day, compute, storage)

Outline

Understood.

FEAT-001 — Control Plane Core (Outline)

Feature ID: FEAT-001
Category: Control Plane
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Operator
Dependencies: None (foundation)

1) Problem

Axion needs an authoritative control layer that records and governs: runs, stages, gates, artifacts, proofs, pins, policies, overrides, releases, and audit history—so the pipeline is deterministic, inspectable, and reproducible.

2) Goals

- Single source of truth for pipeline execution state
- Deterministic provenance (inputs → stages → outputs)
- Stable read/write surface for CLI/UI/CI
- Audit-grade traceability and tamper-evidence
- Namespace isolation (multi-project readiness)

3) Non-Goals

- Full distributed scheduler/cluster orchestration
- Enterprise SSO/SCIM
- Multi-region HA/replication
- Implementing all stages/gates (Control Plane records outcomes)

4) Scope Boundary

In scope

- Run lifecycle state machine
- Stage lifecycle recording
- Gate result recording + evidence pointers
- Artifact + proof registry (hashing + refs)
- Pinning and lineage between runs
- Policy decision + override records
- Release records (immutable metadata + signatures fields)

- Audit log (tamper-evident)

Out of scope

- UI implementation
- Stage implementation logic
- Remote artifact store implementation (only pointers/refs)
- External IdP integration

5) Primary Flows

- **Create Run**: snapshot resolved inputs → create run → queue stages
- **Stage Start/Finish**: record timings → register outputs (artifacts/proofs) → link provenance
- **Gate Record**: store gate outcome + evidence refs + optional trace
- **Pin + Rerun**: attach pins → create rerun with lineage → enforce pinned versions
- **Release**: create immutable release object tied to a successful run and manifest hash

6) Core Objects (conceptual)

- Namespace, Run, StageRun, GateResult
- Artifact, Proof
- Pin
- PolicyDecision, Override
- AuditLog (hash chained)
- Release

7) Persistence & Integrity

- Content hashing for artifacts/proofs
- Hash-chained audit log
- Immutable input snapshot manifest per run
- Referential integrity (no dangling refs)

8) Interfaces (high level)

- Control Plane API used by:
    - CLI runner/orchestrator (run/stage/gate events)
    - Operator UI (read-only + action endpoints)
    - CI/PR checks (read run/gate state, publish results)

9) Determinism Rules (feature-level)

- State transitions are explicit and validated
- No implicit IO registration (every artifact/proof must declare type/schema/version/hash)

- Runs capture environment fingerprint + input snapshot reference

10) Acceptance Targets (high level)

- Can create a run with an input snapshot reference
- Can record stage start/finish and register artifacts/proofs
- Can record gate results with evidence refs
- Can pin versions and create a rerun lineage
- Can produce a release record with manifest hash and signature slots
- Audit log chain validates end-to-end

FEAT-001 — Add-ons (the required 8)

Add-on 1) ID + Registry Entry

- FEAT-001 added to `FEATURE_REGISTRY` with: category=control_plane, priority=P0, status=outlined, dependencies=[]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- pipeline_id + pipeline_definition_ref
- target_profile
- resolved standards/templates/constraints set refs
- operator identity (actor_id)
- stage callbacks (start/finish payloads)
- gate evaluation payloads (results + evidence refs)

Outputs

- persisted CP records for all core objects
- immutable input snapshot manifest per run (hash + pointer)
- provenance links (consumes/produces refs)

State mutations

- Run status transitions
- StageRun status transitions
- Artifact/Proof registration append-only (no silent overwrite)
- Pin attach
- Override approvals

- Release creation (immutable)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (Control Plane scope):

- `CP_INVALID_STATE_TRANSITION` (no)
- `CP_SCHEMA_VALIDATION_FAIL` (no)
- `CP_REF_RESOLUTION_FAIL` (no)
- `CP_WRITE_FAIL` (yes, bounded)
- `CP_HASH_MISMATCH` (no)
- `CP_UNAUTHORIZED` (no)
- `CP_POLICY_BLOCKED` (no)
- `CP_CONCURRENCY_CONFLICT` (yes, retry)

Add-on 4) Security/Privacy/Trust

- Roles: Operator / Reviewer / Auditor (minimum)
- Namespace isolation enforced on every read/write
- Redaction rules applied before storing logs/policy inputs
- Artifact classification enforced on export paths
- Signature fields supported for releases (trust verification hooks)

Add-on 5) Gates + Proof

Gates

- `GATE-CP-01` Schema validation on artifact/proof registration
- `GATE-CP-02` State transition validity
- `GATE-CP-03` Audit chain integrity
- `GATE-CP-04` Release requires required proofs + gate pass set
- `GATE-CP-05` Run captures env_fingerprint + input_snapshot_ref

Proof types

- `PROOF-RUN-SNAPSHOT` (inputs snapshot manifest)
- `PROOF-AUDIT-CHAIN` (audit chain head + verification result)
- `PROOF-RELEASE-ATTEST` (release attestation metadata)

Add-on 6) Tests (minimum set)

- Unit: state machine transitions; hash chain; schema validation; ref resolution

- Integration: stage start/finish → artifact/proof registration; gate recording; pin + rerun lineage; release creation
- Golden: same input snapshot → stable manifest hash; rerun lineage preserved
- Adversarial: circular refs; tampered audit entry; undeclared artifact overwrite attempt; unauthorized access

Add-on 7) Observability

- Events: run_created, run_started, stage_started, stage_finished, gate_recorded, artifact_registered, proof_registered, pin_added, override_requested, override_approved, release_created
- Metrics: CP write latency, registration throughput, gate record throughput, audit verify time
- Logs: structured fields include run_id, stage_id, gate_id, namespace_id, actor_id, error_code

Add-on 8) Docs

- Control Plane overview (objects + lifecycle)
- Operator actions + expected outcomes
- Error codes + remediation
- Data retention + classification behavior
- Release + provenance semantics (what's guaranteed)

FEAT-001 status: Outlined + add-ons defined (ready to lock and then implement).

Operator UI Core

FEAT-002 — Operator UI Core (Run + Gate + Artifact Views)

Feature ID: FEAT-002
Category: Operator UI
Priority: P0
Status: Outline (pre-addons)
Owner Role: Operator
Dependencies: FEAT-001 (Control Plane API + records)

FEAT-002 — Outline

1) Problem

Operators need a single place to **run**, **inspect**, and **debug** Axion pipelines. Without a consistent operator UI, failures are slow to triage, reruns are error-prone, and provenance is hard to verify.

2) Goals

- View runs and their lifecycle status
- See stage timeline + gate outcomes per run
- Open a gate failure and see: rule_id, pointers, diffs, evidence refs
- Browse artifacts produced by a run (type/version/hash/path)
- Trigger “re-run stage” actions against a specific run (via CP)

3) Non-Goals

- Full collaboration (comments/approvals) in v1 (tracked separately)
- Full standards/template authoring UI (read-only links only)
- Full remote artifact retrieval UI (show pointers; fetch handled elsewhere)

4) Scope Boundary

In scope

- Run picker + run list filters
- Run detail page: stage timeline + gate outcomes
- Gate failure viewer panel
- Artifact browser (run-scoped)
- Proof ledger viewer (run-scoped, basic)

- One-click re-run stage control (calls CP)

Out of scope

- Editing standards/templates/constraints
- Advanced diff tooling beyond “rendered + raw + file diff pointer”
- Notifications/subscriptions

5) Primary Flows

- **Open run list** → filter by status/profile/date → open run
- **Inspect run** → timeline view → open stage → open gate result
- **Investigate gate fail** → view pointers/diff/evidence → open referenced artifacts/proofs
- **Browse artifacts** → filter by type → open raw/rendered metadata
- **Re-run stage** → select stage → confirm pinned inputs shown → trigger rerun

6) Core Views (MVP)

- Run List
- Run Detail (timeline)
- Gate Result Detail (failure viewer)
- Artifacts (run-scoped list + detail)
- Proofs (run-scoped list + detail)

7) Data Dependencies (read model)

- Runs, StageRuns, GateResults, Artifacts, Proofs, Pins (from CP)
- Provenance edges (consumes/produces refs)
- Evidence refs (artifact/proof/diff pointers)

8) Determinism Rules (feature-level)

- UI is **read-only truth** from Control Plane (no derived “guessing” state)
- Every UI element maps to a CP object ID (run_id, stage_id, gate_id, artifact_id, proof_id)

9) Acceptance Targets (high level)

- Can navigate from run → stage → gate → evidence artifact/proof
- Can view run artifacts with hash/version/path
- Can trigger re-run stage with pinned versions visible

FEAT-002 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-002 added to `FEATURE_REGISTRY` with: category=operator_ui, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- CP read APIs: runs, stage_runs, gate_results, artifacts, proofs, pins
- CP action API: rerun_stage(stage_run_id or stage_id + run_id)

Outputs

- Operator UI views + local UI state (non-authoritative)
- Deep links to CP object IDs (stable routing contract)

State mutations

- None in CP except via explicit action calls (rerun_stage)
- UI state is ephemeral (filters, open panels)

Add-on 3) Errors (Taxonomy + Retryability)

Add UI/OP errors to `ERROR_CODE_REGISTRY` (operator-ui scope):

- `OPUI_CP_UNAVAILABLE` (retry)
- `OPUI_CP_FORBIDDEN` (no)
- `OPUI_NOT_FOUND` (no)
- `OPUI_RENDER_FAIL` (retry after fallback to raw)
- `OPUI_RERUN_REJECTED` (no; show CP reason)
- `OPUI_EVIDENCE_POINTER_INVALID` (no)

Add-on 4) Security/Privacy/Trust

- Enforce role-based visibility: Operator vs Reviewer vs Auditor (read gates as allowed)
- Respect artifact classification (restricted artifacts masked unless permitted)
- Redaction-safe rendering (never render raw secrets if CP marked redacted/classified)

Add-on 5) Gates + Proof

Gates (UI build readiness)

- `GATE-OPUI-01` UI routes map only to valid CP object IDs (no orphan routes)

- `GATE-OPUI-02` restricted artifact masking enforced in UI

Proof

- `PROOF-OPUI-SNAPSHOT` (optional): screenshot bundle or UI state export for a run (pointer only)
  (If not implementing now: mark as N/A in lock pass.)

Add-on 6) Tests (minimum set)

- Unit: route parsing; rendering components; masking rules
- Integration: CP mock server → run list loads; run detail loads; gate fail opens; artifact detail opens; rerun calls CP
- Golden: known run fixture renders identically (stable fields: ids, statuses, counts)
- Adversarial: missing evidence refs; corrupted diff pointers; restricted artifact access attempt

Add-on 7) Observability

- Events: ui_view_run, ui_open_stage, ui_open_gate, ui_open_artifact, ui_rerun_stage_clicked, ui_rerun_stage_result
- Metrics: page load latency; CP fetch latency; rerun call latency; render error counts
- Logs: structured with run_id/stage_id/gate_id/artifact_id, error_code

Add-on 8) Docs

- Operator UI guide: how to inspect failures, find evidence, rerun stages
- UI object mapping: what each screen reads from CP (by object type)
- Troubleshooting: common OPUI error codes + remediation

FEAT-002 status: Outlined + add-ons defined (ready to lock).

Gate Engine Core

FEAT-003 — Gate Engine Core (Evaluate + Record + Trace)

Feature ID: FEAT-003
Category: Quality / Correctness (Gates)
Priority: P0
Status: Outline (pre-addons)
Owner Role: System
Dependencies: FEAT-001 (Control Plane for GateResult + evidence/proof refs)

FEAT-003 — Outline

1) Problem

Axion needs a consistent way to **execute gate rules deterministically**, capture **why** a rule passed/failed, and attach **evidence pointers** so failures are actionable and auditable. Without a gate engine, “gates” degrade into ad-hoc scripts with inconsistent outputs.

2) Goals

- Deterministic gate evaluation against declared inputs
- Standardized GateResult output (pass/fail/error/skipped)
- Evidence attachment (artifact refs, proof refs, diff refs)
- Trace output for “Gate Trace Mode” (predicate-by-predicate)
- Pluggable gate packs (core + project-specific injection later)

3) Non-Goals

- Full policy engine replacement (policies are broader than gates)
- Autogenerated fix steps (that’s failure playbooks)
- Arbitrary code execution (gates run in a restricted DSL / sandbox)

4) Scope Boundary

In scope

- Gate definition format (minimal DSL or predicate model)
- Gate runtime: load inputs → evaluate → produce GateResult + trace
- Evidence resolver interface (turn “pointers” into refs)
- Deterministic ordering (no nondeterministic iteration)

- Gate pack loading + versioning hooks

Out of scope

- UI for authoring gates
- Distributed execution across workers
- Heavy semantic analysis (keep v1 crisp)

5) Primary Flows

- **Evaluate gates at checkpoint**
  - For stage checkpoint: resolve gate set → evaluate in order/parallel policy → emit results
- **Record outcome**
  - Write GateResult to CP with evidence refs + trace ref
- **Trace**
  - Store predicate trace for a failing (or requested) gate
- **Skip**
  - If applicability says “not active” or upstream lock missing → skipped with reason

6) Gate Definition (MVP capability)

Each gate definition includes:

- `gate_id`, `version`, `description`, `applies_when` (profile/domain/stage)
- `inputs` (artifact types/refs required)
- `rules` (predicates)
- `evidence` mappings (what refs to attach on fail)
- `error_codes` it may emit

7) Execution Model

- Input resolution (from CP refs + workspace pointers)
- Deterministic evaluation:
  - stable iteration order
  - explicit parallelization only when safe
- Output:
  - GateResult object + optional trace blob
- Failure classification:
  - rule failure vs engine error vs missing input

8) Determinism Rules (feature-level)

- No network access during evaluation (unless explicitly allowed, default deny)

- Inputs are immutable snapshot refs (or pinned versions)
- Trace always reproducible for same inputs

9) Acceptance Targets (high level)

- Can evaluate a defined set of gates and persist results to CP
- Can generate predicate trace on demand
- Can attach evidence refs and error codes consistently

FEAT-003 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-003 added to `FEATURE_REGISTRY` with: category=gates, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Gate pack definitions (gate_id/version/applies_when/rules)
- CP refs for required artifacts/proofs
- Execution context: run_id, stage_id, profile, namespace_id
- Optional: “trace requested” flag

Outputs

- GateResult records in CP (status + evidence_refs + error_code)
- Trace blob stored as `trace_ref` (pointer) when produced

State mutations

- Append GateResult entries to CP (immutable records; no silent overwrite)
- Optional trace artifact/proof registration

Add-on 3) Errors (Taxonomy + Retryability)

Add gate engine errors to `ERROR_CODE_REGISTRY`:

- `GATE_ENGINE_INPUT_MISSING` (no; actionable)
- `GATE_ENGINE_RULE_EVAL_ERROR` (no; bug or unsupported rule)
- `GATE_ENGINE_FORBIDDEN_OPERATION` (no)

- `GATE_ENGINE_TIMEOUT` (retry bounded)
- `GATE_ENGINE_CP_WRITE_FAIL` (retry bounded)
- `GATE_ENGINE_PACK_INVALID` (no)
- `GATE_ENGINE_NONDETERMINISM_DETECTED` (no; blocks)

Add-on 4) Security/Privacy/Trust

- Restricted execution: no shell, no network by default
- Gate packs must be versioned; optional signature verification hook
- Evidence pointers must respect artifact classification (don't leak restricted into public contexts)

Add-on 5) Gates + Proof

Gates (meta-gates for gate engine)

- `GATE-GE-01` Gate pack schema validation (all gates well-formed)
- `GATE-GE-02` Determinism enforcement (stable ordering + no forbidden ops)
- `GATE-GE-03` Evidence refs resolvable (no dangling pointers)

Proof types

- `PROOF-GATE-TRACE` (predicate trace blob, schema'd)
- `PROOF-GATE-RUNSET` (summary of evaluated gates + versions + inputs snapshot refs)

Add-on 6) Tests (minimum set)

- Unit: predicate eval correctness; applies_when filtering; stable ordering; evidence mapping
- Integration: evaluate gates against fixture artifacts; write GateResults to CP; trace generation persists
- Golden: same inputs → same GateResult + same trace hash
- Adversarial: malformed gate pack; missing inputs; forbidden operation; nondeterminism triggers

Add-on 7) Observability

- Events: gate_pack_loaded, gate_evaluated, gate_failed, gate_skipped, gate_trace_written
- Metrics: gate eval latency, pack load time, fail rate by gate_id, timeout count
- Logs: run_id, stage_id, gate_id, gate_version, error_code, evidence_count

Add-on 8) Docs

- Gate definition format (fields + examples)
- Evidence pointer rules (what can be attached and how)
- Trace semantics (what's recorded, privacy considerations)
- Troubleshooting guide for gate errors + common fixes

FEAT-003 status: Outlined + add-ons defined (ready to lock).

Artifact Store + Registry

FEAT-004 — Artifact Store + Registry (Content-Addressed Storage)

Feature ID: FEAT-004
Category: Packaging / Distribution (Storage)
Priority: P0
Status: Outline (pre-addons)
Owner Role: System
Dependencies: FEAT-001 (Control Plane Artifact/Proof records)

FEAT-004 — Outline

1) Problem

Axion needs a reliable way to **store, retrieve, and dedupe** artifacts/proofs by hash, while keeping Control Plane records small and authoritative. Without a proper artifact store, bundles are inconsistent, caching is unsafe, and provenance cannot be validated end-to-end.

2) Goals

- Content-addressed storage (CAS): `hash → blob`
- Store and fetch artifacts/proofs via stable `storage_ref`
- Support local store first (filesystem), with remote-store interface later
- Enforce classification-aware storage paths (public/internal/restricted)
- Provide integrity checks on read/write (hash verification)

3) Non-Goals

- Remote artifact store implementation (S3/GCS) in v1
- Full CDN/performance optimization
- Cross-region replication

4) Scope Boundary

In scope

- Local CAS implementation + directory layout
- Store API: put/get/exists/verify
- Reference format (`storage_ref`) and store kind abstraction

- Garbage collection hooks (mark/sweep via CP refs)
- Integrity enforcement (hash match required)

Out of scope

- Remote push/pull (interface only)
- Encryption at rest beyond optional local “encrypted vault” mode (separate feature if needed)

5) Primary Flows

- **Put artifact**
  - receive blob + metadata → compute hash → write to CAS path → return storage_ref
- **Get artifact**
  - storage_ref → read blob → verify hash → return blob/stream
- **Exists/Verify**
  - quick existence checks + full hash verification
- **GC**
  - CP enumerates live refs → store removes unreferenced blobs (policy-driven)

6) Store Layout (MVP)

- Root: `.axion/store/`
- By kind + classification:
  - `blobs/<hash_prefix>/<hash>`
  - `meta/<artifact_id>.json` (optional, small metadata mirror)
- Classification segregation:
  - `public/`, `internal/`, `restricted/` partitions (prevents accidental export leaks)

7) Interfaces (high level)

- `store.put(blob, expected_hash?, classification) -> {hash, storage_ref}`
- `store.get(storage_ref) -> blob`
- `store.verify(storage_ref) -> ok|error(CP_HASH_MISMATCH)`
- `store.exists(storage_ref) -> bool`
- `store.gc(live_hashes, policy) -> report`

8) Determinism Rules (feature-level)

- Hash algorithm fixed by profile (default SHA-256)

- Same bytes produce same storage_ref
- Reads verify hashes always (no “trust reads”)

9) Acceptance Targets (high level)

- Can store and retrieve artifacts/proofs by hash
- Detect tampering via hash mismatch
- Support classification partitioning
- GC can remove orphaned blobs safely

FEAT-004 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-004 added to `FEATURE_REGISTRY` with: category=artifact_store, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Blob bytes (artifact/proof content)
- Optional expected hash
- Classification (public/internal/restricted)
- Namespace/workspace context
- CP artifact/proof registration payload includes returned storage_ref

Outputs

- Content hash
- storage_ref (store_kind + classification + hash path)
- Optional metadata mirror file

State mutations

- Writes blobs to CAS
- GC deletes blobs per policy (only after CP identifies orphans)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (store scope):

- `STORE_WRITE_FAIL` (retry bounded)
- `STORE_READ_FAIL` (retry bounded)
- `STORE_HASH_MISMATCH` (no)
- `STORE_REF_INVALID` (no)
- `STORE_NOT_FOUND` (no)
- `STORE_PERMISSION_DENIED` (no)
- `STORE_GC_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Classification-based partitions enforced by API
- Restricted blobs require explicit permission checks (via CP role context)
- Optional redaction check hook (deny storing known-secret patterns if enabled)
- Integrity verification mandatory on read

Add-on 5) Gates + Proof

Gates

- `GATE-STORE-01` Hash verification on read/write
- `GATE-STORE-02` Classification partition enforcement (no cross-class ref)
- `GATE-STORE-03` GC safety (never delete live hashes)

Proof types

- `PROOF-STORE-INTEGRITY` (store verification report: checked hashes + results)
- `PROOF-STORE-GC-REPORT` (blobs deleted + retained + policy used)

Add-on 6) Tests (minimum set)

- Unit: put/get/exists/verify; ref parsing; classification routing
- Integration: CP registers artifact with storage_ref; retrieval verifies hash; tamper simulation triggers mismatch
- Golden: same blob bytes → same hash + same storage_ref
- Adversarial: invalid ref; wrong expected hash; permission denied; GC with live ref protection

Add-on 7) Observability

- Events: store_put, store_get, store_verify, store_gc
- Metrics: put latency, get latency, bytes stored, dedupe rate, gc deletions
- Logs: storage_ref, hash, classification, namespace_id, error_code

Add-on 8) Docs

- storage_ref format + store layout
- Classification rules (public/internal/restricted)
- Integrity guarantees and failure modes
- GC policy and how to run/verify

FEAT-004 status: Outlined + add-ons defined (ready to lock).

Content-Hash Cache

FEAT-005 — Content-Hash Cache + Incremental Rebuild Planner

Feature ID: FEAT-005
Category: Speed & Incrementality
Priority: P0
Status: Outline (pre-addons)
Owner Role: System
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store for cached outputs)

FEAT-005 — Outline

1) Problem

Axion needs to avoid redoing work when inputs haven't changed. Without a cache + incremental planner, runs are slow, expensive, and operators lose confidence in iteration.

2) Goals

- Compute deterministic **input signature** per stage (content-hash of declared inputs)
- Skip stage execution on cache hit and reuse prior outputs safely
- Plan partial rebuilds: only rerun stages impacted by changes
- Record cache decisions in Control Plane (audit + provenance)
- Support shared-cache later (interface only)

3) Non-Goals

- Cross-machine distributed caching in v1 (shared cache interface only)
- Speculative execution / prefetching
- Machine-learning-based change prediction

4) Scope Boundary

In scope

- Stage input signature computation (declared inputs only)
- Cache key format and storage
- Cache hit/miss decisioning
- Incremental rebuild planner (dependency-based)
- Cache integrity checks (prevent poisoning)

- CP recording: cache_status, reused artifact refs, provenance links

Out of scope

- Remote/shared cache infra (only adapter interface)
- Advanced “semantic caching” (content only in v1)

5) Primary Flows

- **Plan Run**
  - build stage dependency graph → compute impacted set (if partial run) → order stages
- **Before Stage Execute**
  - compute stage input signature → look up cache entry
  - if hit: verify cached outputs hashes exist + match → mark stage skipped/reused
  - if miss: run stage normally, then store cache entry
- **After Stage Execute**
  - record produced artifacts → write cache entry: key → produced refs + proof refs + environment fingerprint (if required)

6) Cache Key (MVP)

Cache key fields (deterministic serialization):

- `namespace_id`
- `stage_id`
- `pipeline_id` (or pipeline version hash)
- `target_profile`
- `input_signature_hash` (hash over ordered list of input refs + their hashes)
- `env_fingerprint` (optional strictness: “strict cache” mode)

7) Cache Entry

- key → `produces_refs[]`, `proof_refs[]`, `gate_summary_ref` (optional), `created_at`, `source_run_id`
- classification constraints (don’t reuse restricted outputs into public profile)

8) Incremental Planner (MVP)

- Inputs: stage graph + “changed items” set (artifacts/templates/standards/constraints/config)
- Output: list of stages to run + reasons
- Rules:
  - if a stage consumes a changed input → stage impacted

    - downstream stages impacted transitively
    - pinned inputs override “latest” resolution (stability)

9) Determinism Rules (feature-level)

- Signatures only include declared inputs (no ambient reads)
- Stable ordering when hashing (sorted by ref ID)
- Cache reuse requires hash verification (integrity gate)

10) Acceptance Targets (high level)

- Identical inputs → stage cache hit reuses outputs
- Changed input → planner reruns only impacted stages
- Cache poisoning/tamper triggers safe failure, not reuse

FEAT-005 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-005 added to `FEATURE_REGISTRY` with: category=incrementality, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Stage IO contract (declares consumes)
- Input refs + hashes (from CP/Artifact store)
- Pipeline graph + stage ordering constraints
- Profile + namespace context
- Optional: list of changed items for partial rebuild

Outputs

- Stage plan (run vs skip + reason)
- Cache entries (key → produced refs)
- CP updates: StageRun.cache_status + reused refs

State mutations

- Writes cache entry on successful stage completion
- On cache hit: CP records reuse (no artifact rewriting)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (cache scope):

- `CACHE_KEY_INVALID` (no)
- `CACHE_LOOKUP_FAIL` (retry bounded)
- `CACHE_ENTRY_CORRUPT` (no)
- `CACHE_HASH_MISMATCH` (no)
- `CACHE_CLASSIFICATION_CONFLICT` (no)
- `CACHE_ENV_MISMATCH` (no; treated as miss)
- `CACHE_STORE_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Never reuse outputs across incompatible classifications/profiles
- Cache entries include classification + profile constraints
- Strict cache mode binds to env_fingerprint to avoid unsafe reuse across toolchains

Add-on 5) Gates + Proof

Gates

- `GATE-CACHE-01` Cache reuse requires hash verification of all reused artifacts
- `GATE-CACHE-02` Classification/profile compatibility enforced
- `GATE-CACHE-03` Input signature includes all declared inputs (coverage check vs IO contract)

Proof types

- `PROOF-CACHE-DECISION` (hit/miss + key + verified hashes)
- `PROOF-PLAN-IMPACT` (impacted stage set + reasons + changed inputs)

Add-on 6) Tests (minimum set)

- Unit: signature hashing stable; impacted set logic; classification rules
- Integration: run twice → cache hit; modify input → only impacted reruns; corrupt cache entry fails safely
- Golden: stable cache key for same inputs; stable plan output for same change set
- Adversarial: missing declared input (IO mismatch); cache reuse with wrong classification; env mismatch handling

Add-on 7) Observability

- Events: plan_computed, cache_hit, cache_miss, cache_reuse_verified, cache_entry_written
- Metrics: cache hit rate, time saved, reuse failures, planner time
- Logs: run_id, stage_id, cache_key, reason, error_code

Add-on 8) Docs

- Cache key definition + strict/non-strict behavior
- Incremental planning rules (what counts as “changed”)
- Safety rules (integrity + classification)
- Troubleshooting cache misses/hits and common failure codes

FEAT-005 status: Outlined + add-ons defined (ready to lock).**

tandards Resolution Engine

FEAT-006 — Standards Resolution Engine (Applicability + Conflict Resolution)

Feature ID: FEAT-006
Category: Standards, Templates, Governance
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Reviewer
Dependencies: FEAT-001 (Control Plane), existing STANDARDS_INDEX + APPLICABILITY_MAP artifacts

---

FEAT-006 — Outline

1) Problem

Axion needs a deterministic way to select the **effective standards set** for a run (and explain why). Without standards resolution, teams get inconsistent constraints, unclear failures, and un-auditable compliance decisions.

2) Goals

- Resolve the applicable standards set for a given run context
- Explain “why applied” for each standard (rule trace)
- Resolve conflicts deterministically (precedence/composition rules)
- Produce a frozen standards snapshot referenced by the run
- Support signed/verified standards references (hooked, not required in v1)

3) Non-Goals

- Standards authoring UI
- Full legal/compliance mapping
- Automatic standards creation

4) Scope Boundary

In scope

- Input documents: STANDARDS_INDEX, STANDARDS_ENTRIES, APPLICABILITY_MAP, CONSTRAINT_PACKS
- Applicability evaluation against run context (domain/type/target/profile/risk class)

- Conflict resolution rules (explicit precedence)
- Output: resolved standards set + explanation trace + snapshot manifest
- CP recording: standards_snapshot_ref linked to run inputs snapshot

Out of scope

- Organization-wide standards marketplace
- Automated remediation generation

5) Primary Flows

- **Resolve standards at run creation**
  - read index + applicability rules → compute applicable set → apply precedence/merge → output resolved set
- **Explain resolution**
  - for each STD: matched rules + context fields + precedence result
- **Freeze snapshot**
  - write resolved set manifest to artifact store → register proof/ref → attach to run

6) Inputs (conceptual)

- Run context:
  - namespace_id, pipeline_id, target_profile, domain tags, stage list, risk class flags
- Standards library:
  - STANDARDS_INDEX (IDs, titles, scope)
  - STANDARDS_ENTRIES (content)
  - APPLICABILITY_MAP (rules)
  - CONSTRAINT_PACKS (locked rulesets)

7) Outputs (conceptual)

- Resolved Standards Set:
  - list of `{std_id, version, hash, scope, precedence_rank}`
- Resolution Trace:
  - “why applied” detail per std + conflicts resolved
- Snapshot Manifest:
  - immutable manifest hash + storage_ref

8) Conflict Resolution (MVP rules)

- Explicit precedence fields:
  - profile precedence (prod > beta > dev) OR configured order
  - constraint packs override general standards where specified
  - composition/import rules must be explicit (no hidden merges)
- If conflict unresolved → hard fail with specific error + pointers

9) Determinism Rules (feature-level)

- Stable rule evaluation ordering
- Stable sorting of resolved set
- Snapshot hash stable for same inputs and versions

10) Acceptance Targets (high level)

- Given same context + library versions → same resolved set + same snapshot hash
- Conflicts produce deterministic outcome or deterministic failure
- “why applied” is queryable per standard

FEAT-006 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-006 added to `FEATURE_REGISTRY` with: category=standards_resolution, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- STANDARDS_INDEX + STANDARDS_ENTRIES (refs)
- APPLICABILITY_MAP rules (ref)
- CONSTRAINT_PACKS active set (refs)
- Run context (profile/domain/tags/risk flags)

Outputs

- standards_resolved_set (data structure)
- standards_snapshot_manifest (hash + storage_ref)
- resolution_trace (pointer)

State mutations

- CP run inputs snapshot includes standards_snapshot_ref
- Artifact/proof registration for snapshot + trace

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (standards scope):

- `STD_INDEX_INVALID` (no)
- `STD_ENTRY_MISSING` (no)
- `STD_APPLICABILITY_RULE_INVALID` (no)
- `STD_CONFLICT_UNRESOLVED` (no)
- `STD_SNAPSHOT_WRITE_FAIL` (retry bounded)
- `STD_SIGNATURE_UNTRUSTED` (no; if verification enabled)
- `STD_REF_RESOLUTION_FAIL` (no)

Add-on 4) Security/Privacy/Trust

- Standards used for a run must be **pinned by hash** in snapshot
- Optional signature verification hook (trust policy) for standards entries
- Resolution trace must not leak restricted content if standard is restricted-classified

Add-on 5) Gates + Proof

Gates

- `GATE-STD-01` Resolved set must be non-empty when required by profile
- `GATE-STD-02` All standards in resolved set have retrievable entries (no missing docs)
- `GATE-STD-03` Conflict resolution trace present for any overridden/conflicting standard
- `GATE-STD-04` Snapshot hash matches resolved set (integrity)

Proof types

- `PROOF-STD-SNAPSHOT` (resolved set manifest)
- `PROOF-STD-TRACE` (why-applied + precedence trace)

Add-on 6) Tests (minimum set)

- Unit: applicability evaluation; precedence ordering; conflict detection; stable sorting
- Integration: resolve from real library fixtures → snapshot stored → CP linked
- Golden: same inputs → identical resolved set + identical snapshot hash
- Adversarial: malformed applicability map; missing entries; circular imports; unresolved conflicts

Add-on 7) Observability

- Events: standards_resolution_started, standards_resolved, standards_conflict_detected, standards_snapshot_written
- Metrics: resolution time, number of standards applied, conflicts per run
- Logs: run_id, namespace_id, profile, std_count, error_code

Add-on 8) Docs

- Standards model (index/entries/applicability/constraints)
- Conflict resolution rules (explicit precedence table)
- “Why applied” trace format
- Troubleshooting unresolved conflicts + common error codes

FEAT-006 status: Outlined + add-ons defined (ready to lock).**

Template Registry + Renderer

FEAT-007 — Template Registry + Renderer (Preview + Placeholder Enforcement)

Feature ID: FEAT-007
Category: Standards, Templates, Governance
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Operator
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store), FEAT-006 (Resolved standards snapshot informs constraints)

FEAT-007 — Outline

1) Problem

Axion needs a deterministic way to manage templates (IDs/versions), render them with run inputs, and enforce placeholder correctness. Without this, generated artifacts drift, placeholder errors slip through, and operators can't preview "before/after" reliably.

2) Goals

- Template registry: ID, version, schema, required placeholders, output mapping
- Deterministic renderer that produces artifacts with stable hashes
- Placeholder enforcement (unresolved placeholders are errors unless explicitly allowed)
- Render preview support (before/after + highlight unresolved placeholders)
- Template linting (structure + placeholder correctness)

3) Non-Goals

- Full WYSIWYG template editor UI
- Advanced templating language features (keep v1 minimal/deterministic)
- Semantic content generation (templates are structure, not "AI writing")

4) Scope Boundary

In scope

- TEMPLATE_INDEX/REGISTRY model + validation
- Template storage and retrieval by version/hash
- Rendering engine:

    - input binding rules
    - placeholder resolution
    - deterministic rendering
- Render outputs registered as artifacts in CP via store
- Preview payload generation for Operator UI

Out of scope

- Multi-language templating
- Auto template creation
- Remote template marketplace

5) Primary Flows

- **Load template set**
    - select versions (pinned or resolved) → validate registry entries
- **Render**
    - bind inputs → render output → validate placeholders resolved → compute hash → store artifact → register in CP
- **Preview**
    - generate diff-friendly “before/after” preview objects with unresolved highlight
- **Lint**
    - validate template structure and placeholder definitions vs schema

6) Template Model (MVP)

Template registry fields:

- `template_id`, `version`, `hash`
- `template_type` (e.g., markdown, json, yaml)
- `schema_version` (for template structure)
- `placeholders[]` (name, type, required, default?, allow_unresolved?)
- `produces` mapping (artifact_type + schema_version + target path rules)
- `constraints` hooks (standards/constraint packs that apply)

7) Rendering Rules (deterministic)

- Stable ordering of placeholder resolution
- Explicit escaping rules (no context-dependent behavior)
- No network access
- Any unresolved required placeholder → hard error
- Output path must be declared (no implicit writes)

8) Outputs

- Rendered artifacts stored in CAS, registered in CP
- Preview object (raw template + rendered + unresolved list + diff pointer)
- Lint report (violations + pointers)

9) Determinism Rules (feature-level)

- Same template hash + same bound inputs snapshot → same output hash
- Placeholder resolution order is stable and explicit
- Preview generation does not mutate outputs

10) Acceptance Targets (high level)

- Render a template to an artifact and register it deterministically
- Fail fast on unresolved placeholders (unless allowed)
- Produce preview payload usable by Operator UI

FEAT-007 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-007 added to `FEATURE_REGISTRY` with: category=templates, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004, FEAT-006]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- TEMPLATE_REGISTRY entries + template files (refs)
- Bound input values (from run inputs snapshot + artifacts/proofs)
- Active standards/constraint packs snapshot refs

Outputs

- Rendered artifact blobs (storage_ref + hash)
- Preview payload (pointer)
- Lint report (pointer)

State mutations

- Store rendered artifacts + preview/lint proofs
- CP registers artifacts/proofs with produces mapping

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (template scope):

- `TPL_REGISTRY_INVALID` (no)
- `TPL_FILE_MISSING` (no)
- `TPL_RENDER_FAIL` (no)
- `TPL_PLACEHOLDER_UNRESOLVED` (no)
- `TPL_SCHEMA_MISMATCH` (no)
- `TPL_OUTPUT_PATH_UNDECLARED` (no)
- `TPL_STORE_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Templates resolved by hash/version and captured in run snapshot
- Enforce classification rules for rendered outputs (inherit from inputs or explicit mapping)
- No secret expansion into previews unless redaction rules allow it

Add-on 5) Gates + Proof

Gates

- `GATE-TPL-01` Registry entry schema valid + template file retrievable
- `GATE-TPL-02` No unresolved required placeholders
- `GATE-TPL-03` Produces mapping matches artifact schema/type
- `GATE-TPL-04` Preview includes unresolved list + pointers (when applicable)

Proof types

- `PROOF-TPL-PREVIEW` (before/after + unresolved markers)
- `PROOF-TPL-LINT` (lint violations + pointers)
- `PROOF-TPL-RENDER-MAP` (template_id/version/hash → produced artifact refs)

Add-on 6) Tests (minimum set)

- Unit: placeholder resolution; escaping; produces mapping; lint rules; deterministic hashing
- Integration: render template → store artifact → register in CP → preview proof created
- Golden: same inputs produce identical output hash; same preview hash (excluding timestamps)
- Adversarial: missing template file; unresolved placeholder; wrong schema; undeclared output path

Add-on 7) Observability

- Events: template_loaded, template_linted, template_rendered, template_render_failed, preview_written
- Metrics: render latency, lint latency, unresolved placeholder count, render fail rate
- Logs: run_id, template_id, version, hash, produced_artifact_count, error_code

Add-on 8) Docs

- Template registry format + produces mapping rules
- Placeholder typing + unresolved policies
- Preview format + how Operator UI consumes it
- Troubleshooting common template errors

FEAT-007 status: Outlined + add-ons defined (ready to lock).**

Proof Ledger

FEAT-008 — Proof Ledger (Schema'd Evidence Objects + Required Fields)

Feature ID: FEAT-008
Category: Observability & Debugging / Quality
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Auditor
Dependencies: FEAT-001 (Control Plane Proof records), FEAT-004 (Artifact Store)

FEAT-008 — Outline

1) Problem

Axion needs a standardized way to store and validate **proof/evidence objects** so gates, releases, and audits can rely on consistent evidence. Without a proof ledger, evidence becomes unstructured logs and screenshots that can't be validated or traced.

2) Goals

- Proof registry + schemas per proof_type
- Required fields enforcement (proof completeness)
- Searchable proof ledger (by run_id, gate_id, proof_type, artifact refs)
- Proof-to-gate and proof-to-artifact linking
- Support tamper-evidence via hashing + CAS storage refs

3) Non-Goals

- Full text search engine deployment (simple indexed queries in v1)
- UI implementation (Operator UI reads via CP)
- Automatic proof generation (owned by producing features)

4) Scope Boundary

In scope

- PROOF_TYPE_REGISTRY (proof_type → schema_version → required_fields)
- Proof validation at registration time
- Proof linkage model (related_refs)
- Query endpoints for proofs (filters)

- Completeness checks for “release-ready” requirements

Out of scope

- Advanced analytics on proofs
- External audit export formats beyond bundle export feature

5) Primary Flows

- **Register proof**
  - producer submits proof payload → validate against schema → hash → store blob → write CP Proof record
- **Query ledger**
  - list proofs for a run/stage/gate → open proof details → follow related refs
- **Release readiness**
  - check required proofs exist (per profile) before allowing Release creation

6) Proof Model (MVP)

Proof fields:

- `proof_id`, `proof_type`, `schema_version`
- `hash`, `storage_ref`
- `namespace_id`, `run_id` (optional but typical)
- `related_refs[]` (gates/artifacts/stages/runs)
- `created_at`
- `fields` (schema’d payload)

7) Determinism Rules (feature-level)

- Proof payload hashing is over a canonical serialized form (stable ordering)
- Schema validation is deterministic
- Proofs are immutable once registered (new proof = new id/hash)

8) Acceptance Targets (high level)

- Invalid proofs are rejected with clear error codes
- Proof ledger queries return consistent results
- Release can enforce presence of required proof types

FEAT-008 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-008 added to `FEATURE_REGISTRY` with: category=proof_ledger, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Proof payload from producers (proof_type, schema_version, fields, related_refs)
- Proof schema registry entries (required fields, field types)
- Namespace/run context

Outputs

- Stored proof blob (hash + storage_ref)
- CP Proof record with related refs
- Query results (filtered lists)

State mutations

- Add proof records (append-only)
- Update run “proof completeness” derived view (optional; or computed on read)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (proof scope):

- `PROOF_TYPE_UNKNOWN` (no)
- `PROOF_SCHEMA_INVALID` (no)
- `PROOF_REQUIRED_FIELD_MISSING` (no)
- `PROOF_FIELD_TYPE_MISMATCH` (no)
- `PROOF_HASH_MISMATCH` (no)
- `PROOF_STORE_WRITE_FAIL` (retry bounded)
- `PROOF_CP_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Proofs inherit classification rules (or explicit classification field)
- Redaction applied before storage for sensitive fields
- Access control: auditors may read, operators may read/write depending on role
- Optional signature fields supported for proofs that require signer identity

Add-on 5) Gates + Proof

Gates

- `GATE-PROOF-01` Proof registration requires schema validation
- `GATE-PROOF-02` Proofs immutable (no overwrite)
- `GATE-PROOF-03` Release-required proofs present (profile-dependent)

Proof types (owned by this feature)

- `PROOF-LEDGER-REGISTRY` (dump of proof types + schemas in effect for a run)
- `PROOF-RELEASE-PROOF-CHECK` (report: required proofs present/missing)

Add-on 6) Tests (minimum set)

- Unit: schema validation; required field checks; canonical serialization hashing
- Integration: register proof → stored in CAS → CP record created → query by run_id/proof_type
- Golden: same proof payload → same hash
- Adversarial: unknown proof_type; missing required fields; tampered stored blob; invalid related refs

Add-on 7) Observability

- Events: proof_registered, proof_rejected, proof_queried, proof_required_check_run
- Metrics: proof registration latency, rejection rate by error_code, proofs per run
- Logs: proof_id, proof_type, run_id, namespace_id, error_code

Add-on 8) Docs

- PROOF_TYPE_REGISTRY format + schema versioning rules
- Proof payload canonicalization rules (hash stability)
- How producers should register proofs
- Troubleshooting proof validation failures

FEAT-008 status: Outlined + add-ons defined (ready to lock).**

Export Bundles

FEAT-009 — Export Bundles (Manifest + Selective Export + Verification)

Feature ID: FEAT-009
Category: Packaging & Distribution
Priority: P0
Status: Outline (pre-addons)
Owner Role: Operator / Reviewer
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store), FEAT-008 (Proof Ledger)

FEAT-009 — Outline

1) Problem

Axion needs a deterministic way to export a run's outputs into a portable bundle with an integrity-checked manifest. Without export bundles, sharing, audits, CI handoff, and reproducible debugging are fragile.

2) Goals

- Produce bundle(s) from a run: thin/full profiles
- Deterministic manifest: contents + hashes + versions + refs
- Selective export: export only a domain/section/category
- Bundle verification: validate manifest + hash tree + required contents
- Support signed bundles (signature slots in v1; enforcement can come later)

3) Non-Goals

- Remote artifact store syncing (push/pull) in v1
- Streaming partial downloads (can be later)
- UI beyond "Export button calls CP action" (UI handled in FEAT-002)

4) Scope Boundary

In scope

- Bundle builder that gathers artifacts/proofs/standards/templates snapshot refs for a run
- Manifest schema + deterministic ordering
- Bundle naming rules (run_id + manifest_hash)
- Selective export filters (by domain/type/section)

- Verification command/process for bundles
- Optional signing metadata fields

Out of scope

- Release channels workflow (RC promotion) beyond exporting
- External registry publishing

5) Primary Flows

- **Build bundle**
  - select run + profile + filters → collect required refs → materialize files → write manifest → compute manifest hash → write bundle
- **Verify bundle**
  - read manifest → verify hashes and required files → output verification report
- **Selective export**
  - apply filter rules to include only matched artifacts/proofs while preserving integrity (manifest still complete for included items)

6) Bundle Contents (MVP)

- `manifest.json` (authoritative)
- `artifacts/…` (materialized from CAS)
- `proofs/…`
- `snapshots/` (inputs snapshot, standards snapshot, template set snapshot references)
- `run/` (run metadata + gate summary)
- `verify_report.json` (optional output)

7) Manifest Schema (MVP fields)

- `manifest_version`
- `bundle_profile` (thin/full/audit/public/internal)
- `run_id`, `namespace_id`, `pipeline_id`, `target_profile`
- `env_fingerprint`
- `created_at` (optional; excluded from hash if needed)
- `contents[]`: list of `{kind, id, type, version, hash, path, classification}`
- `required_sets`: required proofs/gates for this profile (snapshot)
- `signatures[]` (optional slots)

8) Determinism Rules (feature-level)

- Stable ordering for `contents[]` (sort by kind then id then hash)
- Manifest hash computed from canonical serialization excluding nondeterministic fields
- Bundle file layout deterministic

9) Acceptance Targets (high level)

- Bundle export produces a stable manifest hash for same run+profile+filters
- Verify command detects missing/tampered files
- Selective export includes only requested scope while staying internally consistent

---

FEAT-009 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-009 added to `FEATURE_REGISTRY` with: category=export_bundles, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004, FEAT-008]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- run_id + CP queries for artifacts/proofs/gates/snapshots
- Bundle profile + optional filters (domain/type/section)
- Artifact Store reads for materialization

Outputs

- Bundle file (zip/folder) + manifest hash
- Verification report (pass/fail + details)
- CP record linking run → exported bundle (optional registry entry)

State mutations

- Write bundle to export directory/store
- Register bundle as an artifact (optional; recommended) with manifest hash

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (bundle scope):

- `BUNDLE_BUILD_FAIL` (retry bounded)
- `BUNDLE_MANIFEST_INVALID` (no)

- `BUNDLE_HASH_MISMATCH` (no)
- `BUNDLE_CONTENT_MISSING` (no)
- `BUNDLE_FILTER_INVALID` (no)
- `BUNDLE_STORE_WRITE_FAIL` (retry bounded)
- `BUNDLE_VERIFY_FAIL` (no; verification result)

Add-on 4) Security/Privacy/Trust

- Enforce classification on export:
  - public bundle cannot include restricted/internal content
- Redaction policies applied before export (if required)
- Signature slots supported; trust verification hook for signed bundles (optional in v1)

Add-on 5) Gates + Proof

Gates

- `GATE-BUNDLE-01` Manifest integrity (hash matches canonical manifest)
- `GATE-BUNDLE-02` Bundle contents match manifest (all listed present)
- `GATE-BUNDLE-03` Classification rules enforced for profile

Proof types

- `PROOF-BUNDLE-MANIFEST` (canonical manifest + hash)
- `PROOF-BUNDLE-VERIFY` (verification report + results)

Add-on 6) Tests (minimum set)

- Unit: canonical manifest serialization; sorting rules; filter application
- Integration: export bundle from fixture run → verify passes; tamper file → verify fails
- Golden: same run+profile+filters → same manifest hash
- Adversarial: missing artifact in store; invalid filter; restricted content requested in public profile

Add-on 7) Observability

- Events: bundle_build_started, bundle_built, bundle_verify_started, bundle_verified, bundle_verify_failed
- Metrics: bundle build time, bundle size, verify time, failures by error_code
- Logs: run_id, bundle_profile, manifest_hash, included_counts, error_code

Add-on 8) Docs

- Bundle profiles and what each includes
- Selective export filters (supported selectors)
- Verification procedure + interpreting results
- Troubleshooting bundle build/verify errors

FEAT-009 status: Outlined + add-ons defined (ready to lock).**

Release Objects + Signing

FEAT-010 — Release Objects + Signing (Immutable “Release” Record)

Feature ID: FEAT-010
Category: Packaging & Distribution / Governance
Priority: P0
Status: Outline (pre-addons)
Owner Role: Reviewer
Dependencies: FEAT-001 (Control Plane), FEAT-009 (Bundle manifests), FEAT-008 (Proof Ledger)

FEAT-010 — Outline

1) Problem

Axion needs a formal, immutable “release” object that represents an approved kit/bundle with traceable provenance. Without release objects, there’s no canonical “what shipped,” approvals aren’t auditable, and rollback targets aren’t clear.

2) Goals

- Create immutable Release records that reference:
  - source_run_id
  - exported bundle manifest hash
  - gate summary + required proofs satisfied
- Support signature slots and signer identity recording
- Support channels (dev/beta/prod)
- Provide release verification process (check signatures/manifest/proofs)

3) Non-Goals

- Full PKI/key management service (keyring_ref only)
- External registry publishing
- Auto-promotion pipelines (RC→prod) (separate feature)

4) Scope Boundary

In scope

- Release record creation from a successful run

- Channel assignment and immutability guarantees
- Signature capture (structure + storage; verification hook)
- Release verification command/report
- CP enforcement: releases can only be created when preconditions met

Out of scope

- Signature trust policy authoring UI
- Advanced multi-sig workflows beyond “multiple signatures array”

5) Primary Flows

- **Create release**
  - select run_id + bundle manifest hash + channel → validate preconditions → create Release record → store attestation proof
- **Add signatures**
  - append signature entries (signer, alg, key_ref, sig) → freeze after required signers met (profile-dependent)
- **Verify release**
  - verify manifest integrity + required proofs + (optional) signature validity → produce verification report

6) Release Model (MVP)

- `release_id`, `namespace_id`
- `source_run_id`
- `channel` (dev/beta/prod)
- `manifest_hash` (from FEAT-009)
- `env_fingerprint`
- `required_sets_snapshot_ref` (proof requirements snapshot)
- `signatures[]` (signer_id, key_ref, alg, sig, signed_at)
- `attestation_refs[]` (proof refs)
- `created_at`
- `immutable=true`

7) Preconditions (MVP)

- Source run status = succeeded
- Required gates passed (per profile snapshot)
- Required proofs present (per profile snapshot)
- Bundle manifest exists and verifies

8) Determinism Rules (feature-level)

- Release refers to immutable artifacts by hash
- Any change requires a new release_id (no mutation of released content)
- Verification produces deterministic report for same release data

9) Acceptance Targets (high level)

- Can create release only when conditions met
- Release is immutable and audit-traceable
- Verification report detects missing proofs or manifest mismatch

---

FEAT-010 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-010 added to `FEATURE_REGISTRY` with: category=releases, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-008, FEAT-009]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- source_run_id
- bundle manifest hash (or bundle artifact ref)
- channel
- signer identity (when adding signatures)
- required sets snapshot (gates/proofs required for channel/profile)

Outputs

- Release record in CP
- Release attestation proof + verification report proof

State mutations

- Create immutable Release
- Append signatures (until “locked by policy”; then disallow further changes)
- Record audit entries for create/sign/verify actions

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (release scope):

- `REL_PRECONDITION_FAIL` (no)
- `REL_MANIFEST_VERIFY_FAIL` (no)
- `REL_PROOFS_MISSING` (no)
- `REL_GATES_INCOMPLETE` (no)
- `REL_SIGNATURE_INVALID` (no)
- `REL_SIGNATURES_INSUFFICIENT` (no)
- `REL_IMMUTABLE_VIOLATION` (no)
- `REL_CP_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Release creation requires Reviewer role (or higher)
- Signatures captured with signer identity + key_ref
- Optional trust verification hook (namespace trust policy) for signature validation
- Releases inherit classification rules from bundle profile (public/internal/restricted)

Add-on 5) Gates + Proof

Gates

- `GATE-REL-01` Preconditions enforced (run success + manifest verified + required sets met)
- `GATE-REL-02` Release immutability enforced
- `GATE-REL-03` Signature structure valid (and validity if verification enabled)

Proof types

- `PROOF-REL-ATTEST` (release attestation: source run + manifest + env + required sets)
- `PROOF-REL-VERIFY` (release verification report)

Add-on 6) Tests (minimum set)

- Unit: precondition checks; immutability enforcement; signature append rules; verification report
- Integration: create release from fixture run+bundle; attempt invalid release creation fails; verify release
- Golden: verification report stable for same release
- Adversarial: manifest mismatch; missing proofs; tampered signature; attempt to modify immutable fields

Add-on 7) Observability

- Events: release_created, release_signature_added, release_verified, release_verify_failed
- Metrics: releases per channel, verification time, failures by error_code
- Logs: release_id, run_id, channel, manifest_hash, signer_id, error_code

Add-on 8) Docs

- Release lifecycle and immutability rules
- Signature format and trust verification behavior
- Preconditions and required sets concept
- Troubleshooting failed releases/verifications

FEAT-010 status: Outlined + add-ons defined (ready to lock).**

Policy Engine Core

FEAT-011 — Policy Engine Core (Risk Classes + Hard Stops + Overrides)

Feature ID: FEAT-011
Category: Safety & Risk Controls
Priority: P0
Status: Outline (pre-addons)
Owner Role: Reviewer / System
Dependencies: FEAT-001 (Control Plane PolicyDecision + Override records)

FEAT-011 — Outline

1) Problem

Axion needs enforceable safety rules (risk-based hard stops, override workflows, expiry) so the pipeline can run in environments where security/data/integration mistakes are unacceptable. Without a policy engine, “policies” become guidelines instead of enforceable constraints.

2) Goals

- Evaluate policy rules against run/stage context
- Block, allow, or warn deterministically
- Risk classes (auth/data/integrations/exec/network/etc.)
- Override workflow with expiry + approvals (two-person where required)
- Record PolicyDecision + Override in Control Plane for audit

3) Non-Goals

- Full OPA/Rego runtime in v1 (we can use a minimal policy DSL first)
- Enterprise SSO/IdP integration
- Real-time continuous monitoring beyond pipeline actions

4) Scope Boundary

In scope

- Policy definition format (policy_id/version/scope/risk_class/severity)
- Evaluation points:
    - run creation (inputs/classification)
    - stage start (commands/egress)

    - artifact export (classification)
    - release creation (required proofs/signatures)
- Decision outcomes: allow/block/warn
- Override records + enforcement of expiry + approval requirements
- CP audit trail entries for all decisions/overrides

Out of scope

- UI for authoring policies
- External policy marketplaces

5) Primary Flows

- **Evaluate policy at checkpoint**
    - load active policies for namespace/profile → evaluate → write PolicyDecision → enforce outcome
- **Request override**
    - operator creates override request with reason + expiry → reviewer approval required → enforce
- **Enforce hard stop**
    - if blocked and no valid override → stop stage/run deterministically with error codes

6) Policy Model (MVP)

Policy fields:

- `policy_id`, `version`
- `scope` (run/stage/export/release)
- `risk_class`
- `severity` (block/warn)
- `applies_when` (profile/domain/stage)
- `rule` (predicate: inputs/commands/classification/etc.)
- `requires_two_person` (bool)
- `override_allowed` (bool)

7) Decision Recording

PolicyDecision fields (stored in CP):

- policy_id/version, outcome, evaluated_at
- inputs (redacted), evidence_refs (optional)
- override_ref (if used)

Override fields:

- scope + target ref, reason, expires_at
- requested_by, approved_by, approval_time
- two_person_required flag + enforcement

8) Determinism Rules (feature-level)

- Policies evaluated in stable order (by risk_class then policy_id then version)
- Redaction deterministic (same pattern rules)
- Overrides always explicit and time-bound (no permanent silent bypass)

9) Acceptance Targets (high level)

- Block unsafe actions without override
- Allow safe actions with logged decisions
- Overrides require proper approval/expiry and are enforced
- Full audit trail exists for policy decisions and overrides

FEAT-011 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-011 added to `FEATURE_REGISTRY` with: category=policy_engine, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Active policy set refs for namespace/profile
- Run/stage context (run_id, stage_id, profile, commands, egress intents, artifact classification)
- Override requests/approvals (actor identity + timestamps)

Outputs

- PolicyDecision records in CP
- Enforced action outcome (allow/block/warn)
- Override records + linkage to decisions

State mutations

- Write PolicyDecision entries
- Write Override entries
- Update run/stage status when blocking (via CP state machine)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (policy scope):

- `POLICY_BLOCKED` (no)
- `POLICY_RULE_INVALID` (no)
- `POLICY_SET_MISSING` (no)
- `POLICY_DECISION_WRITE_FAIL` (retry bounded)
- `OVERRIDE_NOT_ALLOWED` (no)
- `OVERRIDE_EXPIRED` (no)
- `OVERRIDE_APPROVAL_REQUIRED` (no)
- `OVERRIDE_TWO_PERSON_REQUIRED` (no)

Add-on 4) Security/Privacy/Trust

- Role enforcement: Operator can request overrides; Reviewer can approve; Auditor read-only
- Redact sensitive fields in policy inputs before storage
- Two-person rule supported for high-risk classes (auth/data/integrations)
- Override expiry enforced by runtime checks (not “best effort”)

Add-on 5) Gates + Proof

Gates

- `GATE-POL-01` Policy set resolved and recorded for run (snapshot ref)
- `GATE-POL-02` Blocked actions stop execution unless valid override exists
- `GATE-POL-03` Overrides must have reason + expiry + approval when required

Proof types

- `PROOF-POLICY-SNAPSHOT` (policies in effect for a run)
- `PROOF-POLICY-DECISIONS` (decision log extract for a run)
- `PROOF-OVERRIDE-RECORD` (override request/approval record)

Add-on 6) Tests (minimum set)

- Unit: predicate eval; stable ordering; expiry enforcement; two-person requirement

- Integration: stage start triggers policy; block stops run; override approval allows; expiry re-blocks
- Golden: same context produces same decision record (excluding timestamps in hash inputs)
- Adversarial: invalid policy rule; attempt override when disallowed; approve override with wrong role; missing redaction

Add-on 7) Observability

- Events: policy_evaluated, policy_blocked, override_requested, override_approved, override_denied, override_expired
- Metrics: blocks per risk class, overrides per week, decision latency
- Logs: run_id, stage_id, policy_id, outcome, risk_class, error_code

Add-on 8) Docs

- Risk classes and what they cover
- Policy definition format + examples
- Override workflow + expiry rules
- Troubleshooting blocked runs and override errors

FEAT-011 status: Outlined + add-ons defined (ready to lock).**

Secrets + PII Scanner

FEAT-012 — Secrets + PII Scanner (Inputs/Outputs) + Quarantine

Feature ID: FEAT-012
Category: Security & Privacy
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Auditor
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store), FEAT-011 (Policy Engine for block/quarantine enforcement)

FEAT-012 — Outline

1) Problem

Axion must prevent accidental leakage of secrets/PII through artifacts, logs, proofs, or bundles. Without scanning + quarantine, exports and evidence packs can become a data breach vector.

2) Goals

- Scan declared inputs and produced outputs (artifacts/proofs/logs) for:
  - common secrets (API keys, tokens, private keys)
  - PII patterns (emails, SSNs, phone numbers, etc.) based on policy profile
- Emit findings as structured results with pointers to locations
- Quarantine flagged artifacts/proofs (restrict access + block export/release as configured)
- Support redaction rules for logs/proofs
- Record findings and actions in Control Plane (audit)

3) Non-Goals

- Perfect detection (v1 is best-effort patterns + allowlist/ignore rules)
- Full DLP suite replacement
- Automated secret rotation

4) Scope Boundary

In scope

- Scanner engine with pattern packs (versioned)
- Scan checkpoints:

    - run creation (inputs snapshot)
    - stage finish (produced artifacts/proofs)
    - bundle export (final pass)
- Quarantine mechanism:
    - mark artifact/proof classification restricted + add quarantine flag
    - policy-driven block of export/release
- Finding model: type, severity, confidence, pointers, evidence snippets (redacted)

Out of scope

- External integrations with vaults or secret managers
- ML-based PII classification

5) Primary Flows

- **Scan on input snapshot**
    - scan inputs → findings → policy decides block/warn → record
- **Scan on stage outputs**
    - scan produced artifacts/proofs → findings → quarantine if configured → record
- **Scan on export**
    - scan bundle contents → block export if violations per profile
- **Review findings**
    - Operator UI reads findings list and affected refs (UI itself in FEAT-002)

6) Finding Model (MVP)

- `finding_id`, `namespace_id`, `run_id`, `stage_id?`
- `finding_type` (secret/pii)
- `pattern_id` + `pattern_pack_version`
- `severity` (block/warn/info)
- `confidence` (high/med/low)
- `pointers[]` (file/path + line/offset ranges, or JSON pointer)
- `affected_refs[]` (artifact/proof/log refs)
- `snippet_redacted` (safe excerpt)
- `created_at`

7) Quarantine Rules (MVP)

- If severity=block:
    - mark affected artifacts/proofs as `quarantined=true`
    - classification escalated to restricted
    - policy blocks export/release unless override explicitly allowed
- If severity=warn:

    - record finding; may allow with warning depending on profile

8) Determinism Rules (feature-level)

- Pattern packs are versioned and pinned in run snapshot
- Stable scanning order (file ordering + deterministic regex application order)
- Findings are reproducible for same content and pack versions

9) Acceptance Targets (high level)

- Detect common secrets/PII patterns in artifacts/logs
- Quarantine works and blocks export/release per policy profile
- Findings are recorded with usable pointers and redacted snippets

---

FEAT-012 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-012 added to `FEATURE_REGISTRY` with: category=security_scanning, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004, FEAT-011]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Content blobs for inputs snapshot + produced artifacts/proofs/logs
- Pattern pack definitions (versioned, pinned)
- Policy profile (block/warn thresholds)
- Namespace/run/stage context

Outputs

- Findings records (structured)
- Quarantine actions applied to affected refs
- Proof/report objects for scans

State mutations

- Add Finding records in CP
- Update Artifact/Proof records: classification/quarantine flags
- PolicyDecision entries for block/warn enforcement (via FEAT-011)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (scanner scope):

- `SCAN_PACK_INVALID` (no)
- `SCAN_EXEC_FAIL` (retry bounded)
- `SCAN_CONTENT_READ_FAIL` (retry bounded)
- `SCAN_QUARANTINE_APPLY_FAIL` (retry bounded)
- `SCAN_POLICY_BLOCKED` (no)
- `SCAN_POINTER_GEN_FAIL` (no)
- `SCAN_REDACTION_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Snippets stored must be redacted by default (no raw secret material)
- Findings access is restricted (at least Operator/Reviewer; Auditor read)
- Quarantine escalates classification and blocks public exports
- Allowlist/ignore rules must be explicit and recorded (optional: later)

Add-on 5) Gates + Proof

Gates

- `GATE-SCAN-01` Scan pack pinned and recorded for run
- `GATE-SCAN-02` Block-severity findings prevent export/release unless override exists
- `GATE-SCAN-03` Quarantined refs cannot be included in public bundles

Proof types

- `PROOF-SCAN-REPORT` (scan summary: counts, affected refs, pack version)
- `PROOF-QUARANTINE-REPORT` (what was quarantined and why)

Add-on 6) Tests (minimum set)

- Unit: regex/pattern matching; deterministic ordering; pointer generation; snippet redaction
- Integration: produce artifact with token → scan flags → quarantine applied → export blocked
- Golden: same content + same pack → same finding hashes (excluding timestamps)
- Adversarial: huge file; binary content; false positive allowlist scenario; redaction failure path

Add-on 7) Observability

- Events: scan_started, scan_completed, finding_recorded, quarantine_applied, export_blocked_by_scan
- Metrics: scans per run, findings per type, quarantine count, scan duration
- Logs: run_id, stage_id, finding_type, severity, affected_ref_count, error_code

Add-on 8) Docs

- What gets scanned and when (checkpoints)
- Finding severities and enforcement rules
- Quarantine behavior and how to resolve
- Troubleshooting false positives / scan failures

FEAT-012 status: Outlined + add-ons defined (ready to lock).**

Referential Integrity Engine

FEAT-013 — Referential Integrity Engine (Broken Ref Finder + Graph)

Feature ID: FEAT-013
Category: Quality & Correctness
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Operator
Dependencies: FEAT-001 (Control Plane), Core Model & Contracts (refs), optionally FEAT-003 (Gate Engine to enforce)

FEAT-013 — Outline

1) Problem

Axion artifacts reference each other (specs → screens/endpoints/entities, gates → evidence, templates → outputs). Without a referential integrity system, broken refs accumulate and failures show up late and ambiguously.

2) Goals

- Validate all references across produced artifacts and registries
- Detect and report broken refs with pointers
- Build a reference graph (upstream/downstream) usable by UI
- Provide a “broken ref finder” report per run and per artifact
- Produce deterministic results (stable ordering)

3) Non-Goals

- Full semantic equivalence (this is structural/reference integrity)
- Auto-fixing refs
- Cross-repo linking beyond the run scope (v1 stays within namespace/run)

4) Scope Boundary

In scope

- Reference format standard (ref types: artifact_ref, spec_ref, gate_ref, proof_ref)
- Ref resolver that checks existence + allowed edges
- Graph builder (nodes=objects, edges=refs)

- Reports:
    - broken refs list
    - orphaned objects (unreferenced where required)
- CP storage of integrity reports + graph snapshot pointer

Out of scope

- Visualization UI (UI consumes produced graph)
- Advanced ranking/prioritization (basic severity first)

5) Primary Flows

- **Run integrity check**
    - scan all relevant artifacts → extract refs → resolve → output report
- **Artifact integrity check**
    - on artifact registration (optional hook) → quick ref check
- **Graph generation**
    - produce nodes/edges snapshot for UI dependency inspector

6) Reference Extraction (MVP)

- For schema'd artifacts: extract known fields that contain refs
- For text artifacts: minimal pattern-based ref extraction (explicit syntax only)
- Every extracted ref includes:
    - ref_type
    - ref_target_id
    - source pointer (file/path + line/JSON pointer)

7) Allowed Edges (MVP)

Examples (enforced):

- screen → endpoint (allowed)
- endpoint → entity (allowed)
- template → artifact (allowed)
- gate_result → artifact/proof (allowed)
  Disallowed edges cause a typed violation.

8) Determinism Rules (feature-level)

- Stable scan order (by artifact_id/hash)
- Stable extraction ordering
- Stable report sorting (by severity then source then ref)

9) Acceptance Targets (high level)

- Broken refs detected with pointers
- Graph snapshot produced and queryable
- Gate can enforce “no broken refs” for required profiles

FEAT-013 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-013 added to `FEATURE_REGISTRY` with: category=ref_integrity, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- Artifact registry (all relevant artifacts for run)
- Spec index (if present) and ref ruleset (allowed edges)
- Proofs/gate results (for evidence refs)
- Namespace/run context

Outputs

- Integrity report (broken refs + disallowed edges + orphans)
- Graph snapshot (nodes/edges)
- Optional per-artifact ref index

State mutations

- Register integrity report as a proof/artifact
- Attach summary to run (optional derived fields)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (ref integrity scope):

- `REF_PARSE_FAIL` (no)
- `REF_TARGET_NOT_FOUND` (no)
- `REF_EDGE_NOT_ALLOWED` (no)
- `REF_GRAPH_BUILD_FAIL` (retry bounded)
- `REF_REPORT_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Graph/report must respect classification (restricted nodes masked in public contexts)
- Ref extraction must not store sensitive snippets (pointers only; redacted context optional)

Add-on 5) Gates + Proof

Gates

- `GATE-REF-01` No broken refs (profile-dependent)
- `GATE-REF-02` No disallowed edges
- `GATE-REF-03` Graph snapshot produced for runs that export bundles

Proof types

- `PROOF-REF-INTEGRITY` (report)
- `PROOF-REF-GRAPH` (nodes/edges snapshot)

Add-on 6) Tests (minimum set)

- Unit: ref extraction from schema'd + text patterns; allowed edge enforcement; stable sorting
- Integration: fixture run with known refs → report matches expected; graph snapshot produced
- Golden: same artifact set → same report hash (excluding timestamps)
- Adversarial: circular refs; missing targets; malformed ref strings; restricted nodes in public view

Add-on 7) Observability

- Events: ref_scan_started, ref_scan_completed, broken_ref_found, graph_written
- Metrics: refs scanned, broken refs count, scan duration
- Logs: run_id, artifact_id, broken_count, error_code

Add-on 8) Docs

- Reference syntax rules + examples
- Allowed edges table (ref policy)
- How to interpret integrity reports
- Troubleshooting broken ref failures

FEAT-013 status: Outlined + add-ons defined (ready to lock).**

Coverage Scoring Engine

FEAT-014 — Coverage Scoring Engine (SPEC_INDEX Buckets + Thresholds)

Feature ID: FEAT-014
Category: Quality & Correctness
Priority: P0
Status: Outline (pre-addons)
Owner Role: Reviewer / System
Dependencies: FEAT-001 (Control Plane), SPEC_INDEX artifacts, optionally FEAT-003 (Gate Engine to enforce)

FEAT-014 — Outline

1) Problem

Axion needs an objective way to measure “how complete” a kit/run is across required SPEC_INDEX buckets (screens/endpoints/entities/etc.). Without coverage scoring, teams ship incomplete specs, gates lack clear thresholds, and reviewers can’t compare runs consistently.

2) Goals

- Compute coverage by SPEC_INDEX bucket
- Support required minimum thresholds per profile (dev/beta/prod)
- Identify gaps: missing items, orphan ACs/tests, incomplete fields
- Support weighted coverage (criticality weighting) when provided
- Produce a deterministic coverage report usable by gates and UI

3) Non-Goals

- Semantic “quality of writing” scoring
- Auto-generating missing specs
- Complex ML scoring

4) Scope Boundary

In scope

- Coverage model: buckets + required fields + minimum counts
- Scoring engine: compute percentage + counts + missing list
- Orphan detection:

    - acceptance criteria without tests
    - tests without acceptance criteria (optional)
- CP storage of coverage report + linkage to run
- Gate integration hook (fail if below thresholds)

Out of scope

- UI dashboards (UI consumes report)
- Auto remediation

5) Primary Flows

- **Compute coverage at checkpoints**
    - after spec generation stages → compute report → store proof → record summary
- **Gate enforcement**
    - profile requires thresholds → fail if below minimum
- **Review**
    - reviewer opens report → sees bucket breakdown + missing items

6) Coverage Inputs (MVP)

- SPEC_INDEX artifact(s) produced for run (structured list of items by category)
- Required fields schema per category (e.g., endpoint must have method/path/auth/req/res schema refs)
- Threshold policy per profile (min % or min counts)

7) Outputs (MVP)

Coverage report includes:

- per bucket: total, present, missing, % score
- required minimums + pass/fail
- missing item list with pointers/refs
- orphan lists (AC↔test)
- optional weighting summary

8) Determinism Rules (feature-level)

- Stable ordering of items (by id)
- Stable computation rules (same inputs → same report hash)
- Threshold policy snapshot pinned by run

9) Acceptance Targets (high level)

- Computes correct bucket coverage from SPEC_INDEX fixtures

- Flags missing/orphan items accurately
- Gate can enforce profile thresholds deterministically

FEAT-014 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-014 added to `FEATURE_REGISTRY` with: category=coverage, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- SPEC_INDEX (by category: screen/endpoint/entity/business_rule/component/acceptance_criterion/test_case/flow)
- Required field rules per category (registry)
- Threshold policy per profile (snapshot ref)
- Namespace/run context

Outputs

- Coverage report (proof/artifact) + summary fields (optional)
- Missing/orphan lists with pointers/refs

State mutations

- Register coverage report as proof
- Attach summary to run (optional derived fields)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (coverage scope):

- `COV_SPEC_INDEX_MISSING` (no)
- `COV_SPEC_INDEX_INVALID` (no)
- `COV_POLICY_MISSING` (no)
- `COV_RULES_INVALID` (no)
- `COV_REPORT_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Coverage report respects classification (restricted items masked in public profile)
- Policy snapshots pinned by hash to run (prevents “moving thresholds”)

Add-on 5) Gates + Proof

Gates

- `GATE-COV-01` Coverage meets thresholds for profile
- `GATE-COV-02` No orphan acceptance criteria (profile-dependent)
- `GATE-COV-03` Coverage policy snapshot recorded for run

Proof types

- `PROOF-COV-REPORT` (bucket breakdown + missing/orphans)
- `PROOF-COV-POLICY` (threshold policy in effect for run)

Add-on 6) Tests (minimum set)

- Unit: scoring math; orphan detection; stable ordering
- Integration: fixture SPEC_INDEX → report stored → gate consumes report
- Golden: same SPEC_INDEX → same report hash (excluding timestamps)
- Adversarial: malformed SPEC_INDEX; missing categories; invalid required rules

Add-on 7) Observability

- Events: coverage_compute_started, coverage_computed, coverage_gate_failed
- Metrics: coverage compute time, bucket scores distribution
- Logs: run_id, profile, bucket_scores, error_code

Add-on 8) Docs

- SPEC_INDEX categories and required fields rules
- Threshold policy model (per profile)
- How to read coverage reports and fix gaps
- Troubleshooting common coverage failures

FEAT-014 status: Outlined + add-ons defined (ready to lock).**

Run Diff Engine

FEAT-015 — Run Diff Engine (Artifact Diffs + Semantic Change Classification)

Feature ID: FEAT-015
Category: Observability & Debugging
Priority: P0
Status: Outline (pre-addons)
Owner Role: Operator / Reviewer
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store)

FEAT-015 — Outline

1) Problem

Operators need to know what changed between two runs to debug regressions, validate fixes, and review standards/template upgrades. Without a run diff engine, comparisons are manual and error-prone.

2) Goals

- Compare two runs (A vs B) and compute:
  - added/removed/changed artifacts
  - changed gates/proofs summaries
- Provide artifact-level diffs (raw + structured when possible)
- Classify diffs semantically (format/metadata/content/breaking vs non-breaking)
- Produce a diff report artifact that UI can render

3) Non-Goals

- Full semantic understanding of text meaning
- UI implementation (UI consumes diff report)
- Cross-namespace comparisons (v1 is same namespace)

4) Scope Boundary

In scope

- Run selection and compatibility check (same namespace)
- Artifact set diff (by artifact type + id + hash)
- Optional structured diff for JSON/YAML (schema'd)

- Diff classification rules (heuristic but deterministic)
- CP record diff report as proof/artifact

Out of scope

- Patch application (auto-apply diffs)
- Visual diff editor

5) Primary Flows

- **Compute run diff**
  - select runA/runB → fetch artifact/proof summaries → compute diff sets → generate per-artifact diff pointers → write diff report
- **Inspect change**
  - UI opens report → picks artifact → opens raw/structured diff (stored pointer)

6) Diff Inputs (MVP)

- Artifact lists for each run: `{artifact_id, type, version, hash, path, classification}`
- Optional schema/type info to enable structured diffs
- GateResult summaries (pass/fail counts per gate_id)
- Proof type summaries

7) Diff Outputs (MVP)

Diff report:

- metadata: runA, runB, timestamps, profiles
- artifact changes:
  - added, removed, modified (hash change)
- per modified artifact: diff_ref + classification (format/metadata/content/breaking)
- gate changes summary
- proof changes summary

8) Semantic Classification (deterministic heuristics)

- format-only: whitespace/order-only changes (for canonicalizable formats)
- metadata-only: changes confined to metadata fields
- content: substantive changes
- breaking: changes that violate schema/version contract or change public interface fields (rule-based)

9) Determinism Rules (feature-level)

- Stable ordering of diff lists (by artifact_id)
- Canonicalization rules fixed and versioned
- Same inputs → same diff report hash (excluding timestamps if needed)

10) Acceptance Targets (high level)

- Detect added/removed/modified artifacts accurately
- Produce diffs for modified artifacts
- Classify diffs deterministically

FEAT-015 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-015 added to `FEATURE_REGISTRY` with: category=run_diff, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- runA_id, runB_id
- CP artifact/proof/gate summaries for each run
- Artifact blobs (for modified artifacts) from store (as needed)
- Canonicalization + classification ruleset (versioned)

Outputs

- Diff report (proof/artifact) + per-artifact diff blobs (optional pointers)
- Summary fields for UI (counts)

State mutations

- Register diff report + diff blobs in store
- Link diff report to runs (optional)

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (diff scope):

- `DIFF_RUN_NOT_FOUND` (no)
- `DIFF_NAMESPACE_MISMATCH` (no)

- `DIFF_FETCH_FAIL` (retry bounded)
- `DIFF_ARTIFACT_READ_FAIL` (retry bounded)
- `DIFF_REPORT_WRITE_FAIL` (retry bounded)
- `DIFF_CLASSIFY_RULES_INVALID` (no)

Add-on 4) Security/Privacy/Trust

- Respect classification: restricted artifacts not diffed into public reports
- Diff blobs must not include redacted secret content (use masked diffs when needed)
- Access control: only permitted roles can view restricted diffs

Add-on 5) Gates + Proof

Gates

- `GATE-DIFF-01` Diff report integrity (hashes match source artifacts)
- `GATE-DIFF-02` Classification enforcement (no restricted leakage)

Proof types

- `PROOF-RUN-DIFF` (diff report)
- `PROOF-DIFF-CLASSIFY-RULES` (ruleset version snapshot used)

Add-on 6) Tests (minimum set)

- Unit: set diff logic; canonicalization; classification heuristics; stable ordering
- Integration: diff two fixture runs → expected counts and classifications; write report; UI-ready format
- Golden: same run pair → same report hash (excluding timestamps)
- Adversarial: missing artifact blobs; mixed classifications; malformed JSON diff input

Add-on 7) Observability

- Events: run_diff_started, run_diff_completed, diff_blob_written
- Metrics: diff compute time, artifacts compared count, modified count
- Logs: runA_id, runB_id, counts, error_code

Add-on 8) Docs

- How run diff works (matching rules)
- Classification definitions (format/metadata/content/breaking)
- Limitations (e.g., binary diffs)
- Troubleshooting diff failures

FEAT-015 status: Outlined + add-ons defined (ready to lock).**

Minimal Repro Exporter

FEAT-016 — Minimal Repro Exporter (Smallest Input Set to Reproduce Failure)

Feature ID: FEAT-016
Category: Observability & Debugging
Priority: P0
Status: Outline (pre-addons)
Owner Role: Operator
Dependencies: FEAT-001 (Control Plane), FEAT-004 (Artifact Store), FEAT-009 (Bundle builder components)

FEAT-016 — Outline

1) Problem

When a run fails, operators need a portable “minimal repro” bundle that contains only what’s required to reproduce the failure deterministically. Without this, debugging and handoff require exporting full bundles (slow, noisy, risky).

2) Goals

- Produce a minimal repro bundle scoped to:
  - failing stage(s)
  - failing gate(s)
  - required upstream inputs (transitively) to reproduce
- Include pins and input snapshots needed to make reproduction deterministic
- Provide a deterministic manifest + verification report (same as bundle system)
- Support “stage-level repro” and “gate-level repro” modes

3) Non-Goals

- Automatic failure fixing
- Perfect minimality across all cases (v1 is “safe minimal”: small but guaranteed sufficient)
- Cross-namespace repro bundles

4) Scope Boundary

In scope

- Identify failure target: (run_id + gate_result_id) or (run_id + stage_id)

- Dependency extraction:
    - upstream consumed artifacts/proofs/templates/standards snapshots
    - gate packs/policy snapshots if needed
- Repro manifest generation (subset of run data)
- Repro verification
- CP records linking failure → repro bundle artifact

Out of scope

- UI beyond “export repro” action
- Remote fetching of missing blobs (assumes local store has required blobs)

5) Primary Flows

- **Create repro from failing gate**
    - take gate_result → collect evidence refs + required inputs → expand upstream via consumes graph → create bundle
- **Create repro from failing stage**
    - take stage_run → collect consumed refs + produced partials (optional) + config snapshots → create bundle
- **Verify repro**
    - verify included contents match manifest; report missing blobs

6) What gets included (MVP)

- input snapshot manifest (run inputs)
- standards snapshot, templates snapshot (if relevant to stage/gate)
- gate pack versions + policy snapshot (if gate/policy involved)
- the failing gate result (metadata + trace ref if exists)
- consumed artifacts/proofs required by the failing stage/gate
- minimal stage context metadata (stage_id, pipeline_id, profile)

7) Selection Algorithm (safe minimal)

- Start set = failure target’s evidence refs + consumes refs
- Expand upstream:
    - for each artifact in set, include its producer metadata and its consumed refs until reaching “snapshot boundary”
- Stop conditions:
    - input snapshot reached
    - standards/templates snapshots included
- Ensure inclusion of required runtime metadata:
    - env_fingerprint
    - pinned versions

8) Determinism Rules (feature-level)

- Selection order stable (sort refs)
- Same failure target → same repro manifest hash (excluding timestamps)
- Always includes snapshots needed to prevent “it works on my machine” drift

9) Acceptance Targets (high level)

- Minimal repro bundle builds for a failing gate/stage
- Bundle verify passes
- Included set is smaller than full bundle while remaining sufficient

---

FEAT-016 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-016 added to `FEATURE_REGISTRY` with: category=minimal_repro, priority=P0, status=outlined, dependencies=[FEAT-001, FEAT-004, FEAT-009]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- run_id + failure selector (gate_result_id or stage_id)
- CP provenance (consumes/produces refs) + snapshots refs
- Artifact store blobs for included refs
- Bundle profile = `repro`

Outputs

- Repro bundle + manifest hash
- Verification report
- CP artifact/proof link: failure → repro bundle ref

State mutations

- Register repro bundle as artifact
- Record audit entry for repro export

Add-on 3) Errors (Taxonomy + Retryability)

Add to `ERROR_CODE_REGISTRY` (repro scope):

- `REPRO_TARGET_NOT_FOUND` (no)
- `REPRO_GRAPH_INCOMPLETE` (no)
- `REPRO_BLOB_MISSING` (no)
- `REPRO_BUILD_FAIL` (retry bounded)
- `REPRO_VERIFY_FAIL` (no)
- `REPRO_FILTER_RULES_INVALID` (no)

Add-on 4) Security/Privacy/Trust

- Classification enforcement: repro bundle cannot include restricted items unless requester permitted
- Redaction applied to logs/traces included in repro
- If quarantined refs are required, repro either blocks or produces restricted-class repro bundle (policy-driven)

Add-on 5) Gates + Proof

Gates

- `GATE-REPRO-01` Repro bundle includes required snapshots + pins
- `GATE-REPRO-02` Manifest integrity and verify pass
- `GATE-REPRO-03` Classification rules enforced for repro profile

Proof types

- `PROOF-REPRO-MANIFEST` (repro manifest + hash)
- `PROOF-REPRO-VERIFY` (verification report)
- `PROOF-REPRO-SELECTION` (why each ref included + selection trace)

Add-on 6) Tests (minimum set)

- Unit: selection algorithm; stable ordering; stop conditions; inclusion of snapshots/pins
- Integration: failing fixture run → repro built for gate fail → verify passes → smaller than full bundle
- Golden: same failure target → same repro manifest hash (excluding timestamps)
- Adversarial: missing upstream ref; circular dependencies; restricted artifact included under wrong permissions

Add-on 7) Observability

- Events: repro_build_started, repro_built, repro_verify_started, repro_verified, repro_selection_completed
- Metrics: repro size, refs included count, build time, verify time

- Logs: run_id, target_id, included_counts, manifest_hash, error_code

Add-on 8) Docs

- What “minimal repro” includes and why
- How selection works (safe minimal rules)
- How to use repro bundles to reproduce failures
- Troubleshooting missing blobs / incomplete graphs

FEAT-016 status: Outlined + add-ons defined (ready to lock).**

Error Taxonomy Registry + Normalized Error Objects

FEAT-017 — Error Taxonomy Registry + Normalized Error Objects

Feature ID: FEAT-017
Category: Observability & Debugging / Governance
Priority: P0
Status: Outline (pre-addons)
Owner Role: System / Auditor
Dependencies: FEAT-001 (Control Plane stores error_code on GateResult/StageRun/PolicyDecision)

FEAT-017 — Outline

1) Problem

Axion needs consistent, machine-readable errors across stages, gates, policies, storage, and UI. Without a centralized taxonomy, failures are hard to search, automate, or audit, and operators can't rely on consistent remediation cues.

2) Goals

- Central `ERROR_CODE_REGISTRY` artifact (versioned)
- Normalized error object schema used everywhere
- Retryability classification (retry/no + bounded policy)
- Required fields: severity, domain, category, operator_action, evidence expectations
- Enforce "only registered error codes allowed" (where feasible)

3) Non-Goals

- Auto-remediation system (that's failure playbooks)
- Internationalization of error messages in v1
- Perfect categorization for every edge case (v1 covers core)

4) Scope Boundary

In scope

- Registry format and required fields
- Normalized error payload schema:

    - `error_code`, `message`, `details`, `pointers`, `evidence_refs`, `retryable`
- Validation hooks:
    - stages/gates/policies must emit registered error codes
- UI mapping:
    - consistent display fields (short title + guidance)

Out of scope

- UI implementation details beyond consumption of schema
- Full “playbook” generation (separate feature)

5) Primary Flows

- **Register/update taxonomy**
    - update registry artifact → version bump → pinned into run snapshots
- **Emit errors**
    - component emits normalized error object → CP stores error_code + payload ref
- **Validate**
    - gates check unknown error codes → block/flag depending on profile

6) Error Registry Model (MVP)

Each entry:

- `error_code` (unique, stable)
- `title` (short)
- `domain` (control_plane/store/gate/template/policy/ui/etc.)
- `severity` (info/warn/block)
- `retryable` (yes/no) + `retry_policy` (if yes)
- `operator_action` (what to do)
- `evidence_expected` (what refs/pointers should be attached)
- `introduced_in` (version/date)

7) Normalized Error Object (MVP)

- `error_code`
- `title` (from registry)
- `message` (instance-specific)
- `context` (run_id, stage_id, gate_id, artifact_id)
- `pointers[]` (file/line/json pointers)
- `evidence_refs[]`
- `retryable`

- `timestamp` (not used in hashing unless required)

8) Determinism Rules (feature-level)

- Error registry pinned by hash into run snapshot
- Normalized error object schema stable
- Validation deterministic: unknown codes handled by profile policy

9) Acceptance Targets (high level)

- All major subsystems emit normalized error objects
- Registry validation blocks unknown error codes where required
- Operator UI can display consistent error summaries from stored data

---

FEAT-017 — Add-ons (required 8)

Add-on 1) ID + Registry Entry

- FEAT-017 added to `FEATURE_REGISTRY` with: category=error_taxonomy, priority=P0, status=outlined, dependencies=[FEAT-001]

Add-on 2) Contracts (Inputs/Outputs/State)

Inputs

- ERROR_CODE_REGISTRY artifact (ref)
- Error emissions from subsystems (normalized schema)
- Profile policy: unknown error handling (block/warn)

Outputs

- Validated error objects stored as artifacts/proofs (optional pointer) and linked in CP records
- Registry snapshot reference pinned into runs

State mutations

- CP records store `error_code` + optional `error_payload_ref`
- Registry updates require version bump and are auditable

Add-on 3) Errors (Taxonomy + Retryability)

Errors for this feature itself:

- `ERRREG_INVALID_SCHEMA` (no)
- `ERRREG_DUPLICATE_CODE` (no)
- `ERRREG_UNKNOWN_CODE_EMITTED` (no/warn depending on profile)
- `ERRREG_SNAPSHOT_WRITE_FAIL` (retry bounded)

Add-on 4) Security/Privacy/Trust

- Error payloads must not include secrets; pointers + redacted snippets only
- Registry changes should be approval-gated (Reviewer role) (enforced via process/policy)

Add-on 5) Gates + Proof

Gates

- `GATE-ERR-01` ERROR_CODE_REGISTRY schema valid + pinned for run
- `GATE-ERR-02` Unknown error codes disallowed for required profiles
- `GATE-ERR-03` Error payloads include required fields for their error_code's expectations

Proof types

- `PROOF-ERRREG-SNAPSHOT` (registry in effect for a run)
- `PROOF-ERR-VALIDATION-REPORT` (unknown codes + missing required fields)

Add-on 6) Tests (minimum set)

- Unit: registry validation; duplicate detection; normalized error schema validation
- Integration: subsystem emits error → CP stores → validation gate checks registry
- Golden: registry snapshot hash stable for same registry content
- Adversarial: unknown code emission; missing required fields; secret in payload (blocked)

Add-on 7) Observability

- Events: err_registry_loaded, err_emitted, err_unknown_detected, err_validation_failed
- Metrics: error rate by error_code, unknown error count, validation failures
- Logs: run_id, stage_id, gate_id, error_code, error_domain

Add-on 8) Docs

- How to add new error codes (process + fields)
- Normalized error object schema and examples
- Unknown code policy behavior by profile

- Troubleshooting “unknown error code” failures

FEAT-017 status: Outlined + add-ons defined (ready to lock).**